

This homework has 8 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and unqiusername(s) of your collaborator(s).

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

2. **Fixed-color-sequence paths.**

Let $G = (V, E)$ be an undirected graph with n vertices and m edges, where $m \geq n$. Also, suppose that each edge is colored either black or white. We say that a path $P = (e_1, \dots, e_k)$ of edges has *color sequence* $(c_1, \dots, c_k)$, where each $c_i \in \{\text{black}, \text{white}\}$, if each e_i has color c_i . In this problem, you will design an algorithm that, given two vertices s, t and a desired color sequence $(c_1, \dots, c_k)$, determines in $O(mk)$ time whether there exists a path of length k from s to t with that color sequence. Note that paths in this problem do not need to be *simple*; they can visit vertices or edges more than once.

The graph $G = (V, E)$ is represented via *adjacency lists*, i.e., for each vertex $i \in V$ there is a list of all its neighbors: vertices j such that $(i, j) \in E$.

- (10 pts) (a) Derive a recurrence relation, including base case(s), for this problem. Briefly justify its correctness.
- (6 pts) (b) Give pseudocode implementing an $O(km)$ -time dynamic programming solution to this problem, and justify its running time. You do not need to give a correctness analysis since you did so for your recurrence.

3. **Back to college** Michael drives down to Ann Arbor from his hometown for the start of each school year. Interested in minimizing his commute time, he invents a modular fuel pack for his car for the commute through various towns in Michigan. This fuel pack allows the vehicle to travel a certain distance before refueling. Each town has a service station where a fuel pack can be reloaded: the current pack is simply swapped out for a fresh full one. (Any remaining fuel in the removed pack will not be used by the car.) The goal is to visit the towns in a specified order, while minimizing the number of fuel pack reloads.

We model this problem as follows. The car can travel a distance up to R on a single fuel pack. Starting from town 0, there are n other towns to visit in sequence. For $i = 1, \dots, n$, the distance from the $(i - 1)$ st town to the i th town is $d_i \leq R$. Given R and an array $D = [d_1, \dots, d_n]$ as

input, the goal is to find a *smallest* set $S \subseteq \{1, \dots, n\}$ of towns (specified by their indices) where refueling should occur, so that the car can reach all n towns without running out of fuel at any point. The car starts out with a full fuel pack, and no refueling is needed at the final town.

- (7 pts) (a) Give a greedy algorithm (including pseudocode) that solves this problem. Defer a correctness argument to the next part, but provide everything else involved in “giving an algorithm” here.
- (10 pts) (b) The setup for your correctness proof is as follows. Let OPT be some arbitrary optimal solution. Let $\text{ALG} = i_1, i_2, i_3, \dots, i_k$ be the towns chosen by your algorithm, in their greedily chosen order. If $\text{ALG} = \text{OPT}$, then ALG is optimal, as needed. So suppose that $\text{ALG} \neq \text{OPT}$, and consider the *first* town i_{diff} in ALG that is not in OPT. (Such an i_{diff} must exist because ALG must have at least as many towns as OPT, since OPT is optimal.) So, OPT contains all of $i_1, i_2, i_3, \dots, i_{\text{diff}-1}$, but not i_{diff} .
You will construct another optimal solution OPT' that contains all of $i_1, i_2, i_3, \dots, i_{\text{diff}}$ by modifying OPT, exchanging out some suitable town j for i_{diff} instead.
Describe which town j to exchange for i_{diff} , and prove that OPT' is an optimal solution. In particular, you should prove that OPT' is a valid solution *and* an optimal solution.
- (4 pts) (c) Explain in your own words why the previous part (proving that OPT' is an optimal solution that contains $i_1, \dots, i_{\text{diff}}$) ultimately proves that your algorithm is correct.
- (5 pts) (d) Now, suppose that the price of a fuel pack may differ from town to town. For example, a fuel pack might cost \$8 in the first town, \$3 in the second town, etc. So, the input now also includes an array $C = [c_1, \dots, c_n]$, where c_i is the cost of refueling in the i th town. The goal now is to complete the tour while *minimizing the total cost* of the fuel packs that are purchased along the way.
Michael claims that we should use the same greedy algorithm that you proposed above to solve this problem. Determine whether this claim is true or not.
- If it is true, give a proof using a greedy exchange argument.
 - Otherwise, provide a small counterexample: give a specific input, the output of your greedy algorithm on that input, and a better solution for that input; also, *very briefly* explain where your proof from the previous part fails for this problem.

4. Dynamic programming for dynamic shortest paths.

Graphs that arise in real applications are often not fixed, but can change over time; such graphs are called “dynamic.” (This is a totally different use of the word “dynamic” than in “dynamic programming”. Computer scientists sometimes name things in confusing ways!) For example, roads and intersections can be added to or deleted from the road network, computers can be added to or removed from the Internet, and people can (un)follow each other on social networks. For many problems of interest, there are algorithms for dynamic graphs that are faster than just re-computing answers “from scratch” whenever the graph changes. In this problem you will give such algorithms for shortest-path problems.

Let $G = (V, E)$ be a weighted directed graph with $n = |V|$ vertices, where the weight of each edge (u, v) is denoted $w(u, v)$. (There can be negative-weight edges, but there is no negative-weight cycle in G .) Suppose that we have already computed the all-pairs distance table D of G . That

is, for each vertex pair $u, v \in V$, entry $D(u, v)$ stores the distance (i.e., the length of a shortest path) from u to v in G .

Now, a new vertex v_{new} is added to the graph, together with its incident edges. Let $G_{new} = (V \cup \{v_{new}\}, E \cup E_{new})$ denote the updated graph, where E_{new} consists of all the incoming and outgoing edges for v_{new} . (There is no negative-weight cycle in G_{new} .) We aim to compute the updated distance table D_{new} of G_{new} .

A straightforward approach is to compute D_{new} from scratch using the Floyd-Warshall algorithm, in $\Theta(n^3)$ time. But we want to do better by exploiting the fact that we already have D . In this problem, you will obtain a faster $O(n^2)$ -time algorithm.

- (10 pts) (a) Write expressions for $D_{new}(v_{new}, u)$ and $D_{new}(u, v_{new})$ that hold for all $u \in V$, where the expressions are in terms of the already-known quantities $D(y, z)$ for $y, z \in V$, and $w(y, z)$ for $(y, z) \in E \cup E_{new}$. Evaluating your expressions should take $O(n)$ time for each vertex u , for a total of $O(n^2)$ time. Justify the correctness of your expressions and the evaluation time.

Hint: Consider why the Bellman-Ford algorithm is correct.

- (8 pts) (b) Write an expression for $D_{new}(u, v)$ that holds for all $u, v \in V$, where the expression is in terms of the already-known quantities listed in the previous part, as well as the quantities you computed in the previous part. Evaluating your expression for all $u, v \in V$ should take a total of $O(n^2)$ time. Justify the correctness of your expression and the evaluation time. Observe that by combining the two parts, we have computed the entire new distance table D_{new} of G_{new} in $O(n^2)$ time!

Hint: Consider why the Floyd-Warshall algorithm is correct.

5. MST mashup.

- (5 pts) (a) Consider the following claim: a minimum spanning tree in a graph is *unique* if the edge weights are *distinct* (all different).

Professor Υ presents you with the following bogus “proof” of this claim: *because edge weights are distinct, all choices in Kruskal’s algorithm are completely determined—there are no “ties” between edge weights that the algorithm can choose how to break. So, the algorithm has only one possible output. Because Kruskal’s algorithm is correct for the MST problem, its unique output must be the only minimum spanning tree in the graph.*

Briefly explain what is wrong with this “proof.” In particular, indicate which sentence in the “proof” does not follow from the previous one, and provide a reason for it.

- (8 pts) (b) Give a correct proof for the claim in the previous part.

Hint: Consider two distinct spanning trees T_1, T_2 and prove via an exchange argument that one of them does not have minimum weight.

6. Greedy knapsack filling.

Recall the knapsack problem (sometimes called the “0-1” knapsack problem): you are given n items, where the i th item has weight w_i and value v_i (both positive integers), and a non-negative weight capacity C of the knapsack. An optimal solution is a subset $S \subseteq \{1, 2, \dots, n\}$ of the items having maximum total value

$$\sum_{i \in S} v_i,$$

under the constraint that the total weight is at most the knapsack capacity, i.e.,

$$\sum_{i \in S} w_i \leq C.$$

In this problem, we introduce the *fractional* variant of the knapsack problem, where one may take any fraction $p_i \in [0, 1]$ of any item i . Naturally, a p_i -fraction of item i weighs $p_i \cdot w_i$, and has value $p_i \cdot v_i$. The goal is to maximize the total value of the selected fractional items.

- (2 pts) (a) Briefly explain why the optimal value for the fractional knapsack problem is *at least as large* as that of the original 0-1 knapsack problem (for the same weights, values, and knapsack capacity).
- (5 pts) (b) Consider the following two greedy algorithms for the fractional knapsack problem:
- VALUEGREEDY: While there is still some unused knapsack capacity and at least one unconsidered item, choose an item having the *largest value* (among those not considered yet), and add the largest possible fraction of that item (up to the entire item) that will fit within the remaining knapsack capacity.
 - WEIGHTGREEDY: While there is still some unused knapsack capacity and at least one unconsidered item, choose an item having the *smallest weight* (among those not considered yet), and add the largest possible fraction of that item (up to the entire item) that will fit within the remaining knapsack capacity.

Suppose we run *both* algorithms VALUEGREEDY and WEIGHTGREEDY, and output a best one of their two outputs. Is this a correct algorithm for the fractional knapsack problem? If so, prove it. Otherwise, give an input for which this algorithm's output is not optimal, and show why it is not.

- (10 pts) (c) Consider the following greedy algorithm for the fractional knapsack problem:
- RELATIVELYGREEDY: While there is still some unused knapsack capacity, choose an item having the largest *relative value* v_i/w_i (among those not considered yet), and add the largest possible fraction of that item (up to the full item) that will fit within the remaining knapsack capacity.

In this part, you will prove that the RELATIVELYGREEDY algorithm is a correct algorithm for the fractional knapsack problem. This is notable because we used a more complicated dynamic programming algorithm to solve 0-1 Knapsack, but the fractional variant admits a much simpler greedy algorithm!

The setup for your correctness proof is as follows. Without loss of generality, suppose that the items are in sorted order by relative value $r_i = v_i/w_i$, from largest to smallest (the algorithm considers them in this order). Let $\text{ALG} = p_1, p_2, \dots, p_n$ be the fractions of items $1, 2, \dots, n$ chosen by the algorithm, respectively. (So, $p_i = 1$ if the algorithm chooses all of item i , and $p_i = 0$ if it doesn't choose any of item i .)

Let OPT be an arbitrary optimal solution. If $\text{ALG} = \text{OPT}$, then ALG is optimal, as needed. So suppose that $\text{ALG} \neq \text{OPT}$, and consider the smallest index j such that OPT does not have *exactly* a p_j -fraction of item j .

You will construct another solution OPT' whose fractions of the first j items are exactly $p_1, p_2, \dots, p_j$ by modifying OPT , using an exchange argument.

Describe which fraction(s) of item(s) to exchange, and prove that OPT' is an optimal solution; this should include a proof that OPT' is a *valid* solution.

7. Optional extra credit: Greedily independent.

An *independent set* of an undirected graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ such that no two vertices in S are adjacent, i.e., $(u, v) \notin E$ for every distinct $u, v \in S$. We are interested in a *maximum independent set*, i.e., the independent set with the *largest* number of vertices. (There may be more than one maximum independent set.)

Consider the following greedy algorithm for finding an independent set in a graph.

```

1: function GREEDYIS( $G = (V, E)$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has smallest degree (i.e., the fewest neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all their incident edges, from  $G$ 
7:   return  $S$ 

```

(1 EC pt)

- (a) Give an example of a graph for which the algorithm GREEDYIS **does not** return the maximum independent set. You can assign numbers to the vertices of your graph, and assume that in the case where there are multiple vertices with the smallest degree, GREEDYIS will choose the vertex having the smallest number.

(2 EC pts)

- (b) In this part, you will prove that GREEDYIS returns a maximum independent set when the input graph is a *tree*.

The setup for the proof is as follows. Let $\text{ALG} = v_1, v_2, v_3, \dots, v_k$ be the vertices chosen by your algorithm, in their greedily chosen order, and let OPT be some arbitrary optimal solution. If $\text{ALG} = \text{OPT}$, then ALG is optimal, as needed. So suppose that $\text{ALG} \neq \text{OPT}$, and consider the *first* vertex v_{diff} in ALG that is not in OPT . (Such a v_{diff} must exist because ALG cannot be a proper subset of OPT ; do you see why?) So, OPT contains all of $v_1, v_2, v_3, \dots, v_{\text{diff}-1}$, but not v_{diff} .

You will construct another optimal solution (a maximum independent set) OPT' that has all of $v_1, v_2, v_3, \dots, v_{\text{diff}}$ by modifying OPT , exchanging out some suitable vertex u for v_{diff} instead.

Describe which vertex u to exchange for v_{diff} , and prove that OPT' is both valid *and* optimal.